

Real-time SARIMA-based anomaly detection in operator networks

Pablo Fondo-Ferreiro ^{a,1,*}, Miguel Rodelgo-Lacruz ^{b,1},
 Francisco Javier González-Castaño ^a, Felipe Gil-Castiñeira ^a, Carlos Zamorano-Pinal ^c,
 David Candal-Ventureira ^a

^a *atlanTTic, Information Technologies Group, Universidade de Vigo, 36310, Vigo Pontevedra, Spain*

^b *Centro Universitario de la Defensa, Escuela Naval Militar de Marín, Marín, 36920, Pontevedra, Spain*

^c *Vodafone España SAU, Spain*

ARTICLE INFO

Keywords:

Anomaly detection
 Time-series
 Operator network
 SARIMA

ABSTRACT

The growing complexity of telecommunication networks increases the need for automated monitoring and detection of incidents. The increasing amount of data generated by current information and communication technologies equipment can be used to identify abnormal behavior and detect anomalies. In this paper, we propose a volume-based real-time solution for anomaly detection in operator networks. The proposal is based on a seasonal autoregressive integrated moving average (SARIMA) forecasting of time-series data, and the identification of outliers. We validate the proposed solution using real traffic traces from a nationwide operator's network which we make publicly available. Our experimental results show that the proposed solution outperforms baseline approaches in terms of precision, being able to substantially reduce the amount of false positives, while maintaining the same level of recall.

1. Introduction

The increasing amount of data which is being generated and collected nowadays plays an important role in the monitoring, analysis and optimization of information systems. These data can be arranged in time series of data observations, labelled with the timestamps when they were generated. The detection of unexpected observations in time series, which is known as anomaly detection, has been widely studied in the literature [1]. However, one of the main limitations of most existing solutions for anomaly detection is that they are highly tailored to specific use cases, requiring expert knowledge of the solution and the scenario to fine tune their operation [2].

Anomaly detection in computer networks represents a major challenge for telecommunication operators that seek to minimize operational expenditures and guarantee the service level agreements with their clients [3]. If we only focus on volume-based anomalies, those related to an increase in traffic volume may be caused by denial of service (DoS) attacks, while those related to a decrease in traffic volume may be caused by infrastructure problems (e.g., broken physical links, damaged devices).

In this paper, we propose and evaluate a solution for real-time detection of traffic volume anomalies in an operator's network, based on the analysis of the time series generated by networking devices.

In detail, our main contributions are:

- The detailed proposal of a complete solution for anomaly detection in real time, based on a statistical time-series forecasting mechanism.
- The proposal of a pruning outlier detection for mitigating false positives.
- The evaluation of the proposed solution using real traffic traces provided by a nationwide operator's network.
- The comparison of the proposed mechanism with a baseline approach.

After this introduction, the paper is structured as follows. Section 2 summarizes the related work. Section 3 presents the problem statement and Section 4 introduces the proposed solution. Then, Section 5 presents the results, which are discussed in Section 6. Finally, Section 7 concludes the manuscript.

* Corresponding author.

E-mail addresses: pfondo@gti.uvigo.es (P. Fondo-Ferreiro), mrodelgo@tud.uvigo.es (M. Rodelgo-Lacruz), javier@gti.uvigo.es (F.J. González-Castaño), xil@gti.uvigo.es (F. Gil-Castiñeira), carlos.zamorano@vodafone.com (C. Zamorano-Pinal), dcandal@gti.uvigo.es (D. Candal-Ventureira).

¹ These authors contributed equally to this work.

2. Related work

2.1. Network anomaly detection

The importance of network anomaly detection has been recognized in Zhang et al. [4], Yaacob et al. [5], which refer to the problem of detecting network-level anomalies from indirect measurements as *network anomography* (combining the terms *anomalous* and *tomography*). They describe that anomaly detection at the network-level is performed by analyzing the error from a network traffic prediction and the actual measurement at a given time instant. Their results reveal that ARIMA methods show high fidelity (low false positives and negatives) and robustness to routing changes and missing or corrupted data. Besides, as an advantage of ARIMA methods for anomaly detection, they highlight their capability of detecting the start and end timestamps of the anomaly.

In [5], Yaacob et al. also evaluate ARIMA for anomaly detection in network traffic, focusing on the detection of Denial-of-Service (DoS) attacks. Similarly to the previous work, the ARIMA model is used to estimate the expected traffic pattern and then it is compared with the actual data to identify anomalies. Although their preliminary findings show that abnormal traffic is correctly detected, they elaborate on the need to further explore the mitigation of false positives.

2.2. Traffic prediction

In [6] Cortez et al. discuss and evaluate three methods for network traffic forecasting: a neural network ensemble (NNE) and two relevant time series methods, ARIMA and Holt-Winters. They evaluate their accuracy with experiments on real-world data using different time scales and also different forecasting lookaheads. In their analysis, they also consider the seasonal and double seasonal versions of the time series methods. They obtain the best results overall with the NNE, which is also the fastest technique. When using short timescales (5 min and 1 h) the performance of the ARIMA model is closer to that of the NNE. Nevertheless, they conclude that the former is impractical for online prediction purposes due to its elevated computational complexity. Holt-Winters performs well for daily timescales due to the low number of samples required.

2.3. ARIMA

ARIMA methods have been extensively used for network traffic prediction [7,8]. In [7], Zhani et al. use a neurofuzzy model (α -SNF), an AutoRegressive Moving Average (ARMA) model and an Integrated AutoRegressive Moving Average (ARIMA) model for short-time traffic prediction. They conduct experiments on real network traffic focusing on the analysis of the effect of some parameters (training set size, number of last observations of the throughput needed as inputs for the model, data granularity, variance and packet size distribution) on prediction error.

In Madan and Mangipudi [8], Madan and Mangipudi argue that ARIMA models are only accurate for making predictions on stationary data and therefore not useful for modeling nonlinear time series; but Recurrent Neural Networks (RNN) can accurately predict them. They propose the use of the Discrete Wavelet Transform (DWT) for decomposing the linear and non-linear parts of the time series and analyzing them with ARIMA and RNN models, respectively. They show that the forecast resulting from this combination outperforms the base models if separately used.

2.4. SARIMA

Seasonal ARIMA (SARIMA) methods have also been explored for network traffic prediction [9,10]. In some works such as [9], Yu et al. use a SARIMA model to make traffic predictions in the long term. In detail, they forecast the average daily traffic per month with high precision. In

[10], Teker et al. use SARIMA and Artificial Neural Networks (ANN) to forecast network bandwidth in a Content Delivery Network (CDN). They show that for 2-day forecasts the SARIMA model outperforms the ANN, for 14-day forecasts the ANN performs better and for 8-day forecasts both methods perform similarly.

2.5. Machine learning

Machine learning techniques have also been explored for network traffic prediction:

In [11], Oliveira et al. compare different artificial neural networks (ANNs) for network traffic prediction. According to its results, simpler neural networks, such as RNNs and Multi-Layer Perceptrons (MLPs) can outperform more complex models such as deep learning stacked autoencoders (SAE).

In Nihale et al. [12], The Long Short-Term Memory (LSTM) model, a type of recurrent neural network, is studied for very short-time traffic prediction, achieving low error with real traffic traces.

Different deep learning approaches are applied to network traffic prediction in Vinayakumar et al. [13], including simple RNN, LSTM, gated recurrent units (GRU) and identity recurrent neural networks (IRNN). In experiments with real traffic data LSTMs outperform other RNNs.

2.6. Kalman filtering and ARIMA

ARIMA predictors have been used in previous works for initializing Kalman Filters [14–16], for prediction of wind speed, road traffic state and frame size in Variable-Bit-Rate video traffic.

However, this strategy has not been explored in the context of network traffic prediction for anomaly detection, to the best of our knowledge.

3. Problem statement

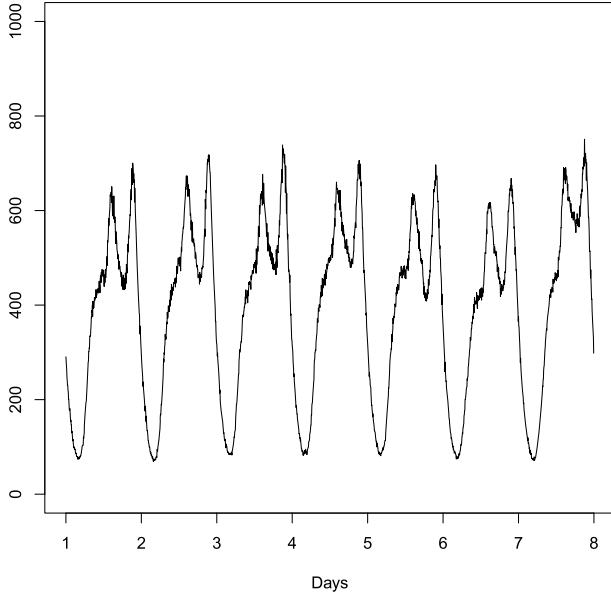
This paper focuses on the problem of real-time detection of anomalies in an operator network. We consider a time-slotted model, where, for every network device at each time slot, the input of the problem is the instantaneous value of a given network metric and the expected output is a classification label among increase anomaly, decrease anomaly and normality (i.e., as opposed to anomaly).

The network metrics considered in this work are aggregated Internet traffic, Virtual Private Network (VPN) traffic, downstream traffic and number of active client sessions. Fig. 1 shows one week of a sample time series for each network metric, with a polling interval of 5 min, which is used in all examples throughout this paper.

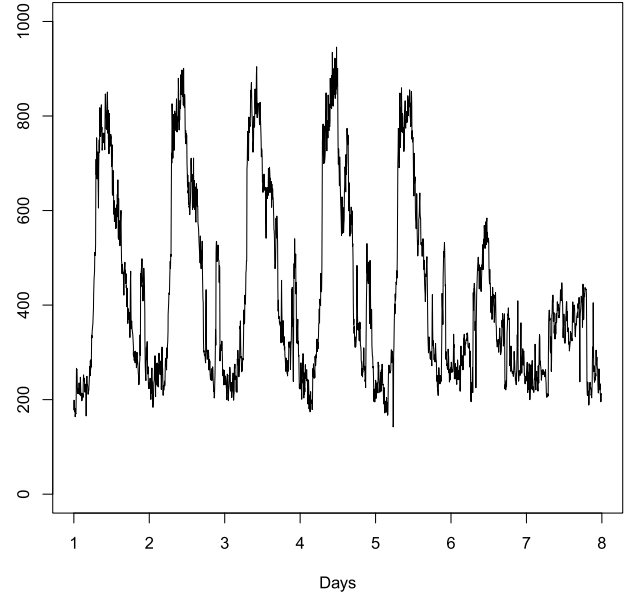
A critical constraint of the problem statement is that detection must take place in real-time. Specifically, the elapsed time of the anomaly detection algorithm must be less than the duration of the time (polling) slot to ensure system stability. Given that an operator network consists of tens of switching centers, each with multiple services, this implies hundreds of network metric measurements to be monitored simultaneously in parallel. Therefore, the models must be spatially and temporally efficient and highly accurate, to avoid generating many false positives that should be manually inspected. It is also desirable for the time series forecasting models to be explainable, in the sense that their internal components and parameters can be interpreted, to understand unexpected temporal behaviors.

4. Proposed solution

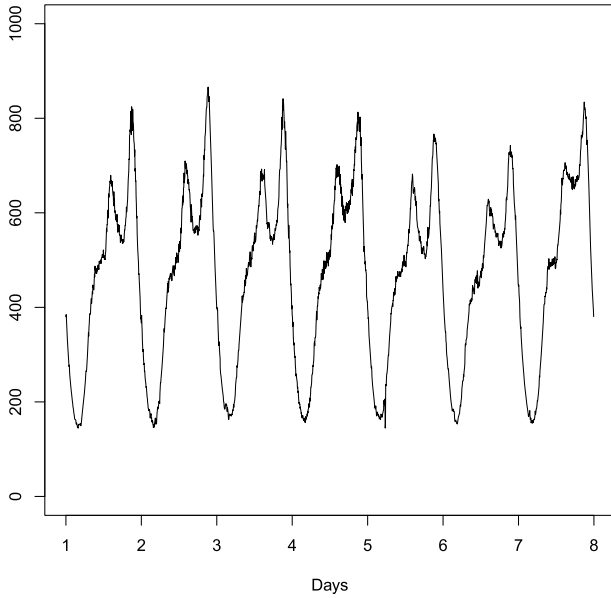
We propose to use well-know time series analysis tools such as Seasonal ARIMA [17] for forecasting the current value of the series. Unlike other approaches based on neural networks, SARIMA models offer robustness and explainability for time series forecasting, as their parameters explicitly represent trend, seasonality, and autocorrelation



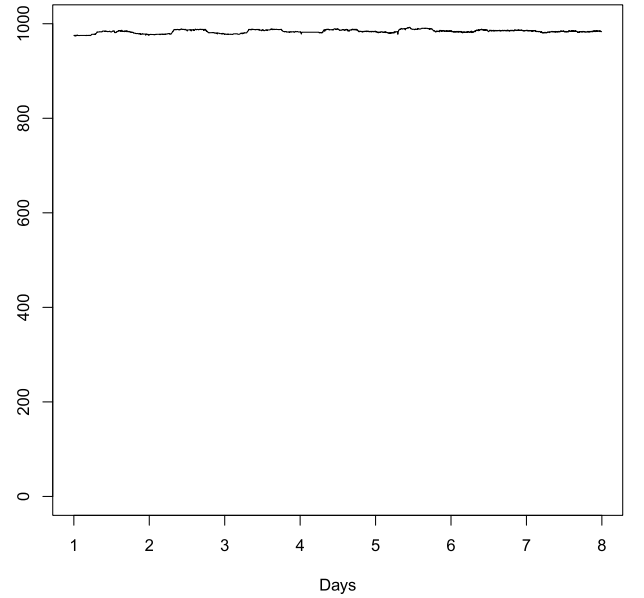
(a) Internet traffic (normalized Mbps)



(b) VPN traffic (normalized Mbps)



(c) Downstream traffic (normalized Mbps)



(d) Number of sessions

Fig. 1. Example time series for the different network metric.

structures in the data [18]. For example, the seasonal period is directly defined as part of the model specification.

A SARIMA model is denoted as:

$$SARIMA(p, d, q)(P, D, Q)[s] \quad (1)$$

where:

- Parameters p, d , and q respectively denote the order of the autoregressive (AR) part, the degree of first differencing, and the order of the moving average (MA) part. They address the non-seasonal

components of the model. The AR component is intended to model the variable of interest using its own p preceding observations. The number of differences d seeks to make the series stationary. The MA component models the error term as a linear combination of error terms occurring q times in the past.

- Parameters P, D , and Q correspond to the seasonal aspects of the model, denoting the order of the seasonal autoregressive component, the degree of seasonal differencing, and the order of the seasonal moving average part. These parameters enable the model to account for correlations between a current value and those from the same season in previous periods.

- Parameter s represents the length of the seasonal period in the data. For example, for a 5 min polling interval, $s = 288$ corresponds to daily seasonality, while $s = 2016$ corresponds to weekly seasonality.

After setting the model orders and the seasonal period, the SARIMA model is fitted to the time series data. This fitting process adapts the coefficients of the model to mimic the provided data as closely as possible. As a result, the fitted model captures the inherent patterns in the time series. Its forecasts identify significant deviations from expected patterns, flagging potential anomalies.

Our proposed solution for addressing the problem comprises six stages, which are explained in detail in following subsections:

- Variance stabilization function:** ARMA-type modeling, including SARIMA, requires stable mean and variance over time. When variance is level-dependent, a variance-stabilizing transformation can decouple data variability from its mean value. The optimal transformation function can either be analytically derived from a functional form expressing the relationship between variance and mean or empirically determined from the data. In this stage, we estimate the optimal transformation using a simple theoretical model based on certain traffic assumptions, and, subsequently, validate this transformation using the data.
- Seasonal Period:** The seasonal period represents the cycle over which patterns in the data repeat. In this stage, we identify the appropriate seasonal period for our data based on observed patterns and domain knowledge.
- Order of Differencing:** The variance of the transformed series is independent from the mean value. However, to achieve full stationarity, the mean value must also be constant over time, without predictable long-term patterns. In this stage, we determine that the resulting series are not stationary in terms of their mean, by calculating the required degrees of both first and seasonal differencing.
- Order of Autoregression and Moving Average:** Once the series are made stationary, a suitable ARMA model is selected. This involves determining the orders of the autoregressive and moving average components for both the seasonal and non-seasonal parts of the model. At this stage, we first analyze the autocorrelation function (ACF) and partial autocorrelation function (PACF) plots to identify potential values. Subsequently, we employ a grid search approach to select the final model.
- Anomaly Threshold:** Once all model parameters are established, a unique model is fitted to each time series. These fitted models predict the current value of the series using information from previous time slots. The predicted values are then compared with the actual ones, and the resulting residuals are checked against a threshold to identify potential anomalies. At this stage, we establish a residual normalization scheme and set an anomaly threshold that balances the trade-off between detecting true anomalies and minimizing false alarms.
- Online anomaly detection:** SARIMA modeling is typically performed with historical data in an offline setting. However, when applying these methods in real-time, several practical considerations arise. The first challenge is related to the parameters and operation of the SARIMA model, which can introduce a substantial number of false positives. The second challenge involves the requirements for historical data and the execution time needed for the simultaneous parallel processing of all models. To address these challenges, we propose a modified anomaly detection method that post-processes the SARIMA model results to mitigate the issue of false positives, as well as implementing the model as a Kalman filter, which also reduces the hardware requirements significantly.

The determination of the variance stabilization function, seasonal period, differencing and model orders, and anomaly threshold is conducted once in an offline setting. Following this setup, the transformed network metrics are fed into a modified SARIMA model in real time,

where predicted values are compared against the threshold for online anomaly detection. The different steps of the proposed procedure are shown in Fig. 2.

4.1. Variance stabilization function

According to the Wold decomposition theorem [19], all stationary data can be approximated using a stationary ARMA model. This implies that ARMA-type modeling requires stationarity of the process, which means that both the mean and variance remain consistent over time (and also that autocovariances are solely dependent on the time lag). To address time-varying volatility, variance can be explicitly modeled with GARCH-type models [20], or, if the variance changes in relation to the level, the original series can be transformed to stabilize variance.

The Box-Cox transformation [21] is a family of power transformations mainly used to normalize data and stabilize variance. The method checks for the smallest standard deviation under the assumption that transformed data has the highest likelihood to be normally distributed when the standard deviation is the smallest. The family of Box-Cox transformations is indexed by parameter λ :

$$f(y) = \begin{cases} \frac{y^\lambda - 1}{\lambda} & \text{if } \lambda \neq 0 \\ \log(y) & \text{if } \lambda = 0 \end{cases} \quad (2)$$

The value of λ indicates the power to which all y data should be raised. It is defined as a continuously varying function with respect to λ , but some specific values correspond to typical power transformations (e.g., if it equals 0.5, the transformation is equivalent to the square root, $\sqrt{y}$). There are some methods for estimating λ from the data [21–24]. They identify not only the optimum value but also a region of reasonable values. If the estimated transformation parameter is close to a common power transformation (log, square root, etc.), it is recommended to use it instead of the optimum, as it will be more meaningful and easier to interpret. A theoretical model or other data analysis that reveals a functional form for the relation between variance and mean, can greatly help to select a suitable power transformation among the reasonable values. In the following subsections, first, the optimal value of the expected λ is estimated analytically from a simple theoretical model and, afterwards, the value is estimated from the data following the Box-Cox method.

4.1.1. Theoretical model

If we omit the effect of the transfers that occur between two sampling intervals, the total number of bytes transmitted in the interval, Y , is equal to the sum of bytes of all transmissions that arrive during the interval, as shown in expression (3), where U is the number of flows transmitted during the interval and X_i is the size of each flow.

$$Y = \sum_{i=1}^U X_i \quad (3)$$

If the number of users is high (the usual case in telco aggregated links), U follows a Poisson distribution, and if, in addition, the flows are independent and identically distributed (regardless of the actual distribution), then Y , the number of transmitted bytes during an interval, follows a compound Poisson distribution. The expected value and the variance of the compound distribution can be derived from the law of total expectation and the law of total variance (expressions (4) and (5)), where μ and σ^2 denote the expected value and variance, respectively, and considering that $\mu_u = \sigma_u^2$, since U follows a Poisson distribution.

$$\mu_y = \mu_u \mu_x \quad (4)$$

$$\sigma_y^2 = \mu_u \sigma_x^2 + \sigma_u^2 \mu_x^2 = \mu_u E(X^2) \quad (5)$$

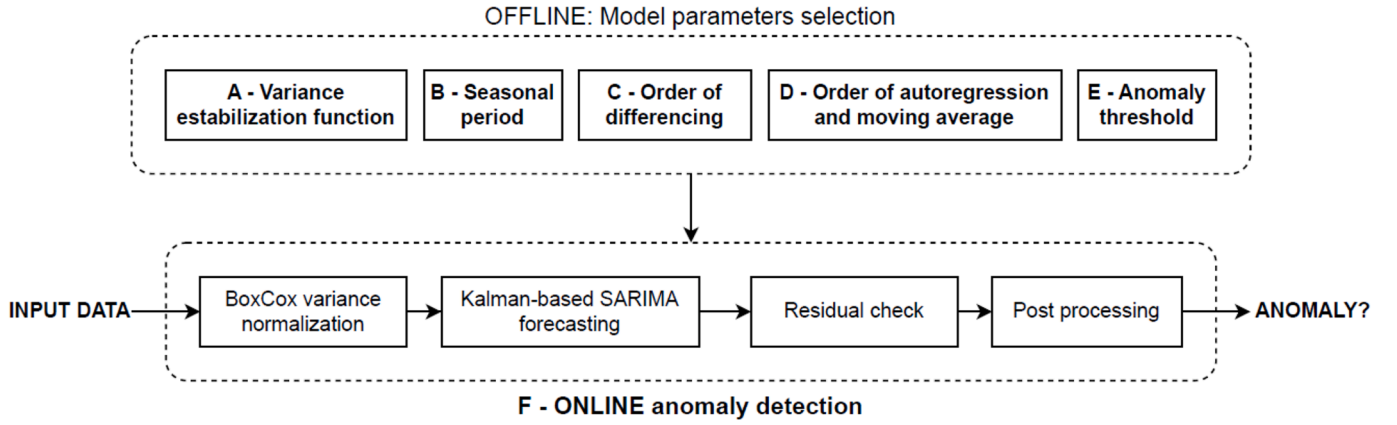


Fig. 2. Steps of the proposed anomaly detection procedure.

Substituting (4) in (5) we can see that the variance of Y is proportional to the mean as shown in expression (6) with $C = E(X^2)/\mu_x$, depending on X , the constant of proportionality.

$$\sigma_y^2 = \frac{\mu_y}{\mu_x} E(X^2) = \mu_y C \quad (6)$$

Note that this implies that the standard deviation of Y , σ_y , grows with the square root of the mean, so traffic fluctuations will be proportionally lower when traffic is aggregated. Other works based on measurements from traces of real Web traffic [25] and the M/G/R PS model [26–28] arrived at the same conclusion about variance behavior: when the link is not heavily loaded, the variance of link utilization increases with mean link utilization. This is natural, because if the flows composing the link traffic are independent of each other, then the variance increases linearly with the number of flows.

If $T = f(Y)$ is a transformation of Y , the first order Taylor approximation around μ_y is given by expression (7).

$$T \approx f(\mu_y) + (Y - \mu_y)f'(\mu_y) \quad (7)$$

Thus, the variance of the transformed values of Y is given by expression (8).

$$\sigma_t^2 \approx \text{Var}(f(\mu_y) + (Y - \mu_y)f'(\mu_y)) = f'(\mu_y)^2 \sigma_y^2 \quad (8)$$

By choosing a Box-Cox transformation function with $\lambda = 0.5$, the first derivative of $f(y)$ is given by expression (9).

$$f'(y) = y^{-0.5} \quad (9)$$

By substituting (9) in (8) and considering that from (6) $C = \sigma_y^2/\mu_y$, the variance of the transformed values is constant (expression (10)).

$$\sigma_t^2 \approx (\mu_y^{-0.5})^2 \sigma_y^2 = C \quad (10)$$

Therefore, if variance is proportional to mean, a square root transformation ($\lambda = 0.5$) should remove the relationship between variability and mean. This is why count processes (Poisson) are usually square rooted for variance stabilization.

Note that we have implicitly assumed in expression (5) that variances are finite, but flow size may be heavy-tailed with infinite variance (Pareto-like), resulting in long-range dependencies and traffic self-similarity [29]. With an infinite variance, the law of total variance and the classical central limit theorem do not hold and the sample mean and variance will not necessary converge. We can expect the ratio of the sample variance and the sample mean to decrease slower than with light tailed traffic when mean traffic and aggregation is increased, since the probability of unexpectedly very heavy events across all the aggregation scales and time windows also grows. Nevertheless, Morris and Lin [25] show that these cases are rare, and despite not being Poisson-distributed, aggregating traffic makes them smooth out mostly as rapidly as with Poisson traffic.

We have also assumed that the number of users is high and the flows are independent and identically distributed. At low loads, when the number of users is low, this assumption may not hold, and the distribution of events can become more uneven and less uniform over time, resulting in increased variability relative to the mean. On the other hand, at heavy loads, saturation can occur, causing the flows to become dependent, and we can expect lower variability of the traffic relative to the mean (in the extreme case, the link operates at maximum capacity and the variance decreases to zero). Both effects contribute to the observed ratio of variance and the mean to increase less than expected by the model and, therefore, the value of λ being higher than predicted. Consequently, we can expect λ values slightly higher than 0.5.

4.1.2. Empirical approach

The empirical analysis to adjust the optimal λ has been performed on the same dataset as in Section 5 using the `boxcox` function from the MASS R package, which calculates profile log-likelihoods. Fig. 3 depicts the progression of the log-likelihood function for λ values from -2 to 2, considering downstream (green), Internet (purple), and VPN (blue) network metric time-series.

As expected, the log-likelihood function for downstream and Internet network metric reaches its maximum around λ values between 0.5 (square root transformation) and 1 (no transformation), consistently across all series, aligning with the results of the theoretical model. Other methods (such as Guerrero [22]) and our own empirical tests exhibit lower optimal λ values closer to 0.5.

The results for VPN traffic network metric measurements show lower and more inconsistent optimal λ values. This behavior can be attributed to the nature of VPN traffic, which typically represents enterprise-related activities with higher correlation patterns (e.g., work schedules, periodic backups). These factors introduce long-range dependencies and result in less smooth aggregation. Additionally, VPN time-series data tend to lower traffic volume and generally smaller differences between the maximum and minimum values.

The results for the session network metric are inconclusive. While the model from Section 4.1.1 holds (since the number of users, U , follows a Poisson distribution), this might be due to the minor variation in the series level (the minimum rarely drops below 95% of the maximum). The series is almost flat and the transformation is practically irrelevant, so the likelihood profile does not discriminate λ well.

Box-Cox was applied to stabilize level-dependent variance rather than to enforce exact normality. In real-time operational settings, the stability and simplicity of using a single λ across all network metrics outweigh the marginal statistical gains obtained by metric-specific tuning, provided that anomaly-detection performance is not materially affected. A compound-Poisson traffic model motivates $\lambda \approx 0.5$, which yields constant first-order variance after transformation. Although the profile likelihood varies across network metrics—lower and less stable for VPN

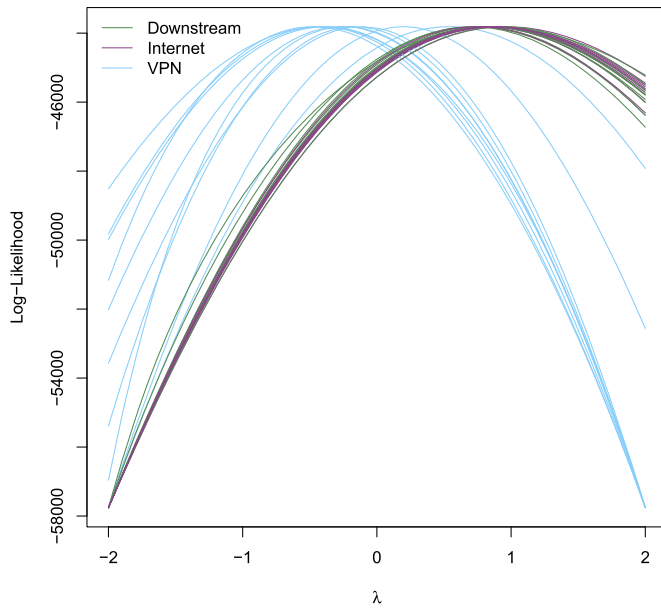


Fig. 3. Empirical analysis to adjust optimal λ .

traffic, and essentially uninformative for the nearly flat session network metric—sensitivity analyses over a range of plausible λ values show that the forecasting performance remains consistent across choices of λ (see Section 5 for additional results). We therefore adopt a fixed $\lambda = 0.5$ for all network metrics, ensuring methodological consistency and computational simplicity without measurable performance degradation.

4.2. Seasonal period

4.2.1. Theoretical model

The seasonal period, denoted as s in a SARIMA model, accounts for recurring patterns that are associated with a fixed time span, such as daily or weekly cycles. Typically, s is chosen based on domain knowledge or apparent periodic patterns observed during exploratory data analysis. In our context, we have two potential seasonal periods: a daily cycle (288 samples for a five-minute interval) and a weekly cycle (2016 samples for a five-minute interval). This is due to the established daily pattern and distinct behavior observed during weekends and holidays in the average levels.

4.2.2. Empirical approach

As anticipated, the exploratory data analysis of our time series (Fig. 1) and ACF (Fig. 4) clearly indicate daily and weekly periodicity. Due to the constraints of limited historical data in Section 5 (2 weeks) and computational capacity, a daily period was selected as the seasonal period rather than a weekly period. However, our analyses using a weekly period yielded similar or even less favorable results compared to those obtained with a daily one.

4.3. Order of differencing

4.3.1. Theoretical model

As described above, the Box-Cox function transforms the time series so that their variance becomes independent from their mean value, but stationarity also requires that the mean does not change over time. In general, a stationary time series will have no predictable patterns in the long-term. Its time plots will show the series to be roughly flat with constant variance and the ACF will drop to zero relatively quickly. Fig. 4 shows the (a) ACF and (b) PACF for the Internet traffic illustrated in Fig. 1(a). The seasonality of the series is obvious, without ruling out the

existence of additional level changes and trends, so the mean is clearly not stationary.

If the mean of the time series is not constant, differencing or seasonal differencing (between an observation and the previous observation from the same season) can remove (or reduce) polynomial and seasonal trends respectively, and the ARMA model becomes an ARIMA or SARIMA model. Sometimes it is necessary to take both a seasonal difference and a first difference to obtain stationary data, but, in practice, it is almost never necessary to go beyond second-order differences [18]. Note there is a degree of subjectivity in selecting which differences to apply, so different analysts may involve different choices.

4.3.2. Empirical approach

One way to determine more objectively whether differencing is required is to use a unit root test. These are statistical hypothesis tests of stationarity that are designed for determining whether differencing is required. The Kwiatkowski-Phillips-Schmidt-Shin (KPSS) test null hypothesis [30] is that the data has a stationary root against a unit-root alternative. Then a sequence of differencing and KPSS tests can be performed to obtain the lowest number of differences required to pass the test at the specified level.

The aforementioned process is carried out in R by the function `ndiffs`. A similar function for determining seasonal differencing is `nsdiffs`. In our case, both functions return 1, suggesting we should perform a seasonal difference and a first difference.

4.4. Order of autoregression and moving average

4.4.1. Theoretical model

In the process of identifying appropriate values for q , Q , p , and P , an analysis of the ACF and PACF plots is usually conducted. These plots may display significant spikes indicative of the appropriate orders. For example, $\text{ARIMA}(p, d, 0)$ data tends to exponentially decaying or sinusoidal ACF with a prominent spike at lag p in the PACF. Conversely, an $\text{ARIMA}(0, d, q)$ model has an exponentially decaying or sinusoidal PACF with a significant spike at lag q in the ACF. These behaviors extend to the Seasonal ARIMA components P and Q and apply to the seasonal lags in the PACF and ACF. Nevertheless, if both p, P and q, Q are strictly positive, determining the correct values can be more challenging. In our case, the plots (Fig. 4) suggest both strictly positive p and q values with a higher number of AR components. This calls for more sophisticated selection strategies such as trial and error or grid search.

The trial and error method begins with initial values for p, q, P , and Q . The model is then fitted and values are adjusted based on performance until satisfactory results are achieved. Each model candidate (defined by the values of p, d, q, P, D, Q) has parameters that are estimated. It is possible to employ a Maximum Likelihood Estimation (MLE) for this purpose. MLE identifies parameter values that maximize the probability of obtaining the observed data, yielding results similar to least squares estimates. Comparison of performance among various fitted models is often facilitated by a loss function, such as the Akaike's Information Criterion (AIC) or Bayesian Information Criterion (BIC). AIC penalizes the model that is fit with the number of parameters to be estimated, often selecting the model with the minimum AIC value as the best choice for forecasting and anomaly detection.

In the grid search approach, a set of candidate values for p, q, P and Q are specified, and the ARIMA model is fit for all possible combinations of candidate values.

4.4.2. Empirical approach

The `auto.arima` function in R uses a variation of the Hyndman-Khandakar algorithm [31], which combines unit root tests, minimization of the MLE and AICc (small-sample corrected AIC) to obtain an ARIMA model. The default procedure uses some approximations to speed up the search, so it is possible to miss the minimum AICc model due to these approximations, or because of the use of a stepwise

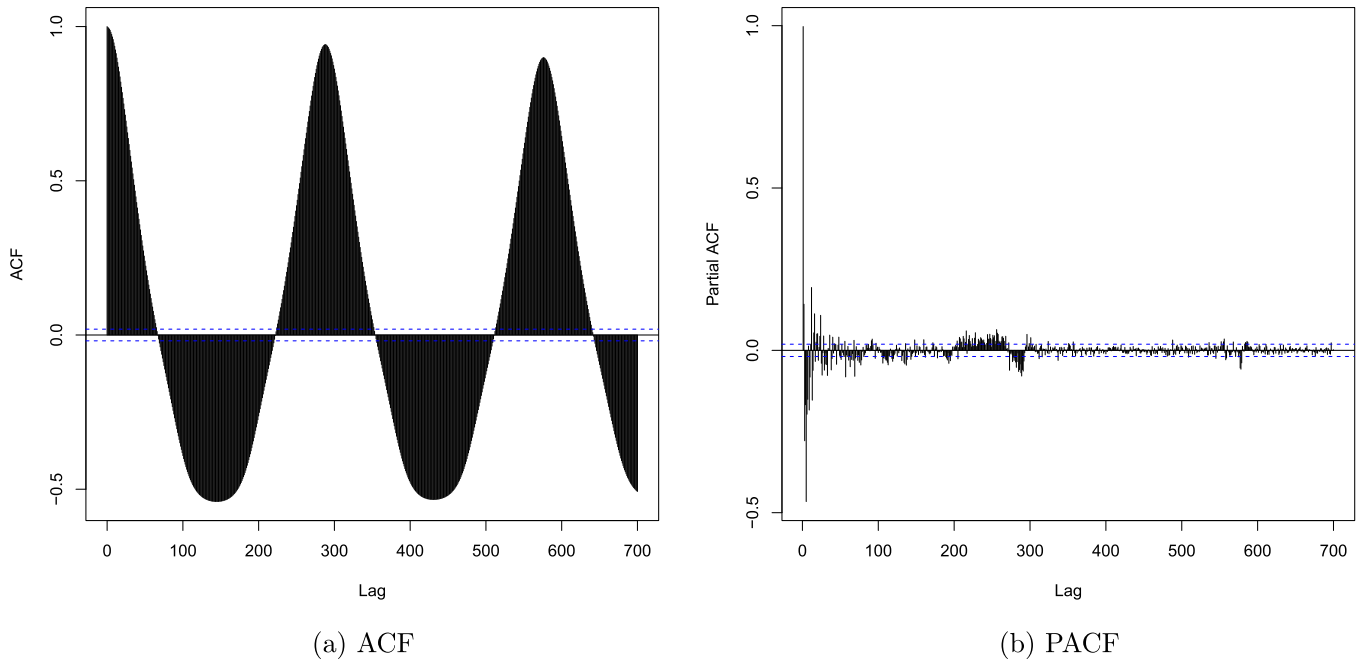


Fig. 4. ACF and PACF for an Internet traffic time series.

procedure. R function `auto.arima` saves time and effort compared to manual trial and error methods, and can also improve the accuracy of the predictions.

In our case, we seek to identify a suitable model that can represent all series pertaining to a specific network metric. Given the multitude of links present in the operator's network and the complexity of estimating SARIMA models compared to regression models, it becomes computationally intensive to formulate an individual model for each time series. Moreover, the automatic selection of models using R function `auto.arima` without human oversight could potentially result in inappropriate model selections, leading to unforeseen errors under certain unexpected conditions. However, considering that the series are derived from identical or highly similar underlying processes, it is logical to assume that they might exhibit similar behaviors and could be aptly represented by a common model. Furthermore, it is plausible that the process of modeling one series could be enhanced by insights gleaned from the others. Therefore, in our approach, we have employed a combination of trial and error, along with grid search, to discover a universally applicable model for each network metric.

The `auto.arima` R function was also run on a set of 36 time series to obtain the model selected by the Hyndman-Khandakar algorithm. For the traffic network metric plots of our dataset in Section 5, none of the models selected by the algorithm had $p > 5$ or $q > 1$. In the seasonal part, no AR or MA components were selected, beyond the differentiation, which may be influenced by the depth of the historical data (2 weeks). Therefore, $ARIMA(5,1,1)(0,1,0)$ [288] was selected as a candidate model. Subsequently, the AIC of the best model of each individual series identified by `auto.arima` was calculated against the AIC of the candidate model, obtaining very similar values and, in some cases, even better with the common model. On the other hand, network metrics related to sessions have a different behavior with much flatter and more stable graphs. In most cases, the optimal model selected by `auto.arima` was $ARIMA(0,1,1)(0,1,0)$ [288], and therefore was selected as the candidate model. Again, the AIC of the best model of each individual series was calculated against the AIC of the candidate model, obtaining very similar values in those cases where they did not coincide.

Finally, the ACF of the residuals was plotted to visually inspect the randomness of the residuals. Ideally, these should be randomly distributed with zero mean. R function `checkresiduals` aids in this pro-

cess, by generating a time plot of the residuals, and the corresponding ACF and histogram. Fig. 5 shows the time plot of the residuals, the ACF and the histogram for a sample time series of an Internet traffic network metric. We can observe that the residuals are normally distributed and a strong correlation for a lag of 288 samples, as expected.

Despite the process not being purely stationary due to long-range dependencies and traffic self-similarity, the residuals appeared reasonably close to white noise, indicating a valid model.

4.5. Anomaly threshold

4.5.1. Theoretical model

Given the unique mean values of each time series, an anomaly threshold, based on the deviation of the residuals, is employed to determine whether they signify an anomaly. Specifically, the residuals are normalized by subtracting the mean of the latest period and dividing it by the corresponding standard deviation. The absolute value of these normalized residuals is subsequently compared against a pre-defined threshold. If the value is greater than this threshold, an anomaly is identified.

If the residuals were perfectly normal, thresholds of 3 or higher should be used, as 0.03% of non-anomalous values fall outside this range in a normal distribution, which would generate almost a daily false alarm for each time series with normal traffic. Although the residuals are approximately Gaussian, Fig. 5 shows that they exhibit a mildly leptokurtic shape (peaked center) with slightly heavier tails than a theoretical Gaussian distribution, a deviation commonly observed in real network traffic and time-series modelling due to the presence of occasional heavy-tailed events [32]. For instance, for the example displayed in Fig. 5, the residual distribution presents an excess kurtosis of approximately 2.32 and an almost zero skewness, indicating a symmetric but noticeably more peaked and heavy-tailed distribution than a Gaussian distribution. Consistently, a Lilliefors (Kolmogorov-Smirnov) test rejects normality at standard significance levels, confirming that the deviations from Gaussian behaviour are statistically significant in this example. Consequently, suitable thresholds must be higher than in the ideal normal case. In fact, as shown in Section 5, our empirical analysis observes detectable anomalies even at thresholds close to 10σ , which would be virtually impossible under true normality—for reference, the

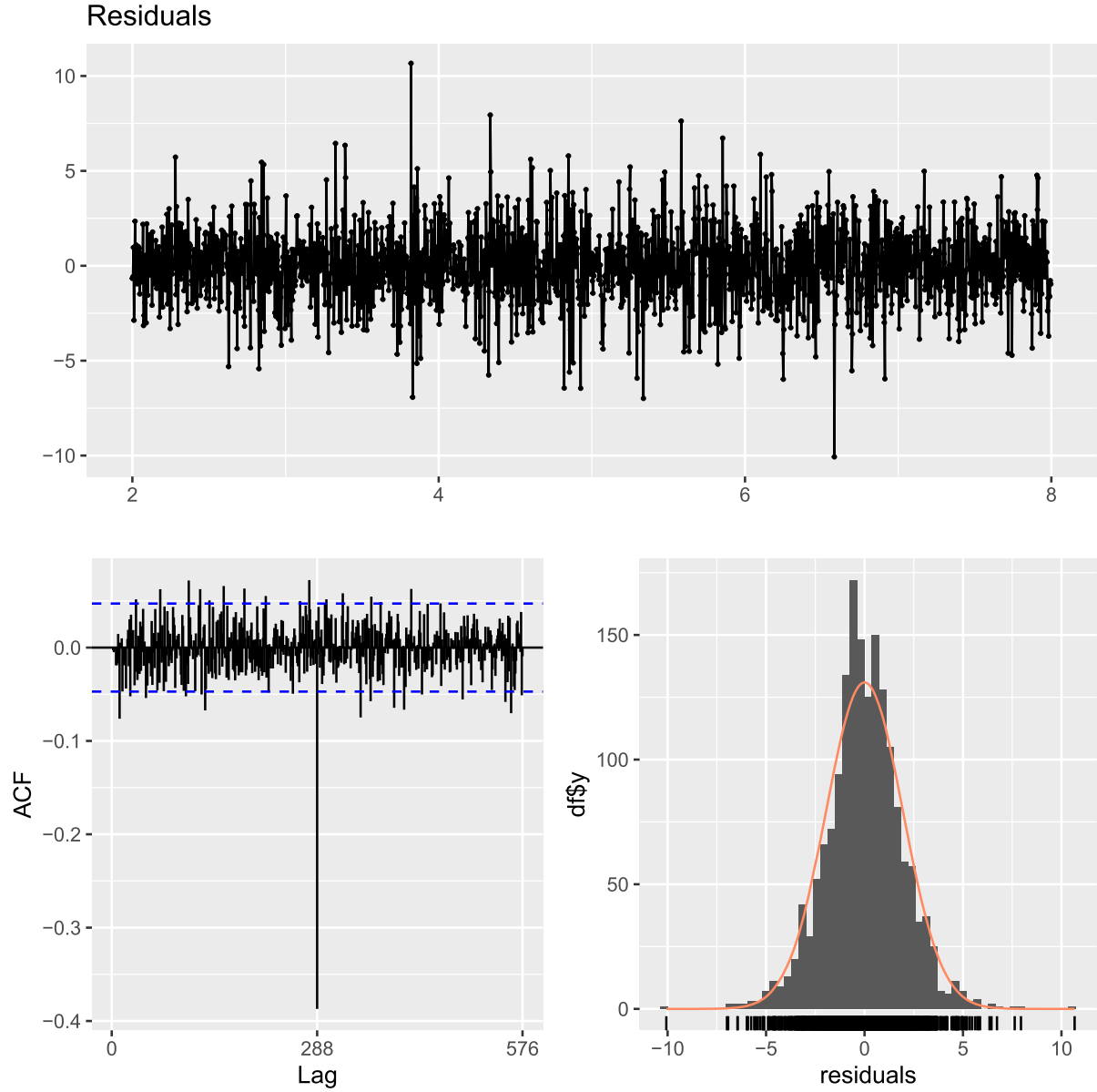


Fig. 5. Time plot of the residuals, ACF and histogram for an example time series of an Internet traffic network metric.

probability of exceeding $\pm 10\sigma$ in a Gaussian distribution is negligible (approximately 7.6×10^{-24}). Thus, despite their near-Gaussian appearance, the residuals contain rare but meaningful extremes that justify evaluating a wide range of thresholds when calibrating the anomaly detector.

4.5.2. Empirical approach

The determination of the optimal threshold is carried out in [Section 5](#), based on the evaluation results.

4.6. Online anomaly detection

Once all model parameters have been established, a unique SARIMA model is fitted to each time series. The transformed network metrics are then fed into the model in real time, where normalized residuals are compared against the threshold to identify potential anomalies in real time. Depending on the sign of the normalized residual, the anomaly

will be labelled as an increase or decrease anomaly. Hereafter, we refer to this anomaly detection algorithm, which uses the SARIMA model directly, as *SARIMA*.

However, this simple procedure introduces a non-negligible number of false positives, since an abnormal value is considered for calculating the predicted values for multiple samples, as shown in [Sections 5](#) and [6](#). In order to increase the precision of the proposed algorithm, we have also considered an improved anomaly detection procedure, in which we consider two additional criteria for determining if a given residual should be considered an anomaly or not: if there was an anomaly of the opposite sign in the samples corresponding to the previous $\max(P, D, Q)$ seasonal periods we do not consider the current sample and none of the following $\max(p, d, q)$ values as anomalies, if they are of opposite sign; if there was an anomaly of the same sign in any of the previous $\max(p, d, q)$ samples, we do not consider the current sample as a new anomaly. In the results, we refer to this more sophisticated version of the anomaly detection for the SARIMA model as *SARIMA-pruned*. [Fig. 6](#) shows the pseudocode of this algorithm.


```

sarima_pruned(time_series, frequency, p, d, q,
              P, D, Q, detection_threshold) {

  // Initialize output variables
  down_anomalies[1..$|residuals$|] = FALSE
  up_anomalies[1..$|residuals$|] = FALSE

  // Get residuals
  raw_residuals = residuals(SARIMA(order=(p, d, q),
                                   frequency=frequency,
                                   seasonal=(P, D, Q),
                                   data=time_series))

  // Normalize residuals
  residuals = (raw_residuals - mean(raw_residuals))
              / std_dev(raw_residuals)

  // Determine anomalies
  valid_down[1..$|residuals$|] = TRUE
  valid_up[1..$|residuals$|] = TRUE
  for i in 1..$|residuals$| {
    if (residuals[i] < -detection_threshold) {
      if (valid_down[TRUE]) {
        down_anomalies[i] = TRUE
      }
      // Avoid mirror anomalies
      for (x in 1..max(P, D, Q)) {
        valid_up[i + frequency $ \cdot x] = FALSE
      }
      // Avoid consecutive anomalies
      for (x in 1..max(p, d, q)) {
        valid_down[i + x] = FALSE
      }
    } else if (residuals[i] > detection_threshold) {
      if (valid_up[i]) {
        up_anomalies[i] = TRUE
      }
      // Avoid mirror anomalies
      for (x in 1..max(P, D, Q)) {
        valid_down[i + frequency $ \cdot x] = FALSE
      }
      // Avoid consecutive anomalies
      for (x in 1..max(p, d, q)) {
        valid_up[i + x] = FALSE
      }
    }
  }

  return (down_anomalies, up_anomalies)
}

```

Fig. 6. Pseudocode of the *SARIMA-pruned* algorithm.

Traditionally, SARIMA models are applied in an offline environment over a complete time series. However, it is also possible to estimate the coefficients incrementally with each new observation. This is known as adaptive or online SARIMA model. Even though R's Arima function provides an argument for supplying an old model, it is necessary to employ a loop that readjusts and updates the ARIMA model with each new sample. Thus, for each new observation, it is required to provide the function with a historical data record, which conveys spatial complexity due to the necessity of maintaining these data or querying them from the database in real time. Additionally, in ARIMA models, the coefficients are adjusted using the Maximum Likelihood Estimation (MLE) method, which is computationally costly for these models.

On the other hand, a SARIMA model can be considered a special case of a state-space model. A Kalman filter can be used to estimate a state-space model and, therefore, a SARIMA model. The Kalman filter is a recursive algorithm, using a process of prediction and updating, and it can be executed in real time only using the current input measurement, the previously computed state, and its uncertainty matrix, without requiring any additional information. Furthermore, it is computationally efficient, making it very suitable for real-time applications.

At this stage, to make the real-time analysis of all network metric anomalies across all switching centers computationally viable, we have employed the KalmanRun function from R's stats package to implement an SARIMA state-space model, obviating the need to store the history of each network metric and providing substantial computational efficiency. The acceleration of this method is demonstrated in Section 5.4.

5. Results

In this Section, we evaluate the proposed SARIMA-based approaches for addressing the problem and compare their performance with a baseline approach consisting in directly using the last value as the prediction for the next one. This naïve strategy is the most basic forecasting approach considering the strong correlation between contiguous samples. In the first experiment, we evaluate the quality of the predictions using the SARIMA model. Note that in this case we are only evaluating the quality of the forecasted values of the time series and thus we only compare the SARIMA with the baseline, since no anomaly detection is being analyzed. The second experiment evaluates the capability of detecting anomalies by introducing synthetic anomalies in the traffic traces. The third experiment evaluates the anomaly detection capabilities on real traffic traces containing real anomalies. Finally, we also measured the execution time of implementations with both the ARIMA function and the Kalman filter. For the implementation and analysis, we used the R forecast package [31].

The dataset used for the experimental evaluation consists in real traffic data from a nationwide network of a telecommunication operator, which has been anonymized and scaled so that the values of each network metric range from 0 to 1000. For the first two experiments, we used a total of 48 different time series from different network metrics and switching centers of the operator network. As previously said, the polling interval was set at 5 min, enabling sufficient data granularity for identifying events and responding to them without generating excessive network traffic due to SNMP (Simple Network Management Protocol) requests. The distribution of the series in network metrics includes 10 VPN series, 10 Internet series, 10 downstream series and 18 series corresponding to sessions. In each time series, we select a time window of 7 days to generate the model, and then ran the step-by-step predictions and anomaly identification for the following 7 days. Thus, the first 7 days can be considered a training set and the following 7 ones a test set. For the third experiment, 16 real anomalies have been manually identified by the operator's staff in a set of 14 time series. We followed a similar procedure as in the first two experiments using this subset, but using the whole time series (more than 30 days) instead of limiting the test set to 7 days. The anonymized dataset has been released under an open license and is publicly available in Fondo-Ferreiro et al. [33].

All traffic-related network metrics (internet, downstream, and VPN) display daily seasonality, meaning valleys and peaks depending on time of day. They vary greatly between their maximum and minimum values (the minimum can be as low as 10% of the maximum) and, under normal conditions, are smooth, showing neither steep descents nor ascents, high variability or ripple (although they are slightly more pronounced for VPN traffic). In contrast, the network metrics related to sessions are also smooth but largely flat (the minimum rarely drops below 95% of the maximum). The vast majority of anomalies, which this work aims to detect, consist of sudden drops in traffic or number of sessions, which persist for a while (minutes to a few hours) and finish with an equally abrupt increase once the issue is solved (typically, a hardware failure in a local node).

In the first experiment, we evaluated the quality of the prediction using the Mean Absolute Percentage Error (MAPE). In the last two experiments, to evaluate the anomaly detection capabilities of our proposal, we evaluated the precision, recall and F1-score of the approaches for a given threshold. For calculating the numbers of true positives (TP), false positives (FP) and false negatives (FN), we have used the anomaly time window approach as in Lavin and Ahmad [34]. For each real anomaly,

Table 1

Prediction quality (MAPE metric) for the training set using the baseline and SARIMA approaches using different λ values.

Strategy	Min	1st Qu	Median	Mean	3rd Qu	Maximum
Baseline	0.01802	0.03460	2.35249	2.92814	2.80593	17.28904
SARIMA ($\lambda = 0.1$)	0.02534	0.05916	1.79503	2.70717	2.27520	18.67487
SARIMA ($\lambda = 0.2$)	0.02327	0.05779	1.77878	2.71044	2.24430	18.94755
SARIMA ($\lambda = 0.3$)	0.02249	0.05689	1.77515	2.71977	2.21817	19.27055
SARIMA ($\lambda = 0.4$)	0.02209	0.05626	1.75752	2.73221	2.18667	19.64146
SARIMA ($\lambda = 0.5$)	0.02179	0.05580	1.72487	2.75286	2.15427	20.05817
SARIMA ($\lambda = 0.6$)	0.02156	0.05545	1.72434	2.77779	2.13594	20.54590
SARIMA ($\lambda = 0.7$)	0.02138	0.05518	1.71760	2.81429	2.12600	21.15474
SARIMA ($\lambda = 0.8$)	0.02124	0.05497	1.68936	2.84950	2.12307	21.82896
SARIMA ($\lambda = 0.9$)	0.02112	0.05479	1.68674	2.89489	2.12497	22.60964

Table 2

Prediction quality (MAPE metric) for the test set using the baseline and SARIMA approaches using different λ values.

Strategy	Min	1st Qu	Median	Mean	3rd Qu	Maximum
Baseline	0.01645	0.04586	2.26107	2.89146	2.73696	16.92158
SARIMA ($\lambda = 0.1$)	0.01924	0.06878	1.71978	2.76877	2.30450	16.28889
SARIMA ($\lambda = 0.2$)	0.01789	0.06745	1.68056	2.76890	2.21827	16.47488
SARIMA ($\lambda = 0.3$)	0.01700	0.06657	1.66990	2.77579	2.20254	16.70730
SARIMA ($\lambda = 0.4$)	0.01640	0.06597	1.66761	2.78641	2.19016	16.98781
SARIMA ($\lambda = 0.5$)	0.01598	0.06554	1.66793	2.80593	2.22815	17.31414
SARIMA ($\lambda = 0.6$)	0.01567	0.06522	1.67057	2.82717	2.20185	17.62444
SARIMA ($\lambda = 0.7$)	0.01544	0.06498	1.67281	2.85475	2.19176	17.83833
SARIMA ($\lambda = 0.8$)	0.01527	0.06480	1.65752	2.88677	2.19976	18.35555
SARIMA ($\lambda = 0.9$)	0.01513	0.06465	1.65918	2.92979	2.20841	18.95298

we considered a time window of 1 h. Then we added one TP if one or more anomalies were detected during the time window. If no anomalies were detected during the time window, we added one FN. Every anomaly detected outside a time window was considered a FP.

Recall that we are not dealing with instantaneous anomalies, but with anomalies lasting for some time. We are mostly interested in anomalies caused by failures in the system, which are reflected in the time series as a decrease anomaly in the starting slot and an increase anomaly after the traffic level recovers to normal values. Therefore, we evaluated the detection of decrease and increase anomalies.

5.1. Prediction quality

Table 1 shows the prediction quality using the MAPE metric [35] for the training set using the proposed SARIMA approach with different λ values, and its comparison with the baseline approach. Similarly, Table 2 shows the prediction quality using the MAPE metric for the test set using the proposed SARIMA approach with different λ values and its comparison with the baseline approach. In both the training and test sets, the SARIMA model achieved similar MAPE values across the tested λ values and consistently outperformed the baseline, thus showing its better forecasting capabilities. We can also observe that increasing λ from 0.1 to 0.9 slightly increases the mean MAPE achieved, while the effect is generally the opposite for the median, first and third quartile values, indicating a more consistent predictive performance for the majority of the series under higher λ values. This suggests that the theoretically motivated choice of $\lambda = 0.5$ would attain a trade-off among the different statistics.

5.2. Anomaly detection capabilities: Synthetic anomalies

As previously described, the test set used for evaluation includes 7 days. Each day in the provided dataset corresponds to 288 samples, so the test set has 2016 samples. We have introduced a synthetic anomaly in each traffic trace. The duration of the synthetic anomaly is uniformly chosen between 1 and 287 samples (i.e., less than one day) and the starting position is selected so that the whole anomaly is contained in

the test set. Each sample affected by the anomaly is reduced a given percentage of its actual value. The magnitude of the anomaly is given by this percentage.

Fig. 7 shows the precision of the decrease and increase anomaly detection approaches, for a synthetic anomaly of a 20% reduction using different detection threshold values. As we could expect, the precision of the approaches grows when increasing the detection threshold up to a certain point (5 for the decrease anomalies, and 6 for the increase anomalies), since the number of FPs decreases faster than the number of TPs. However, further increasing the detection threshold above this point slightly reduces the precision, since the decrease in the number of TPs is more significant than the improvement achieved by reducing the FPs. The basic SARIMA approach obtains the lowest precision for any detection threshold. The SARIMA-pruned approach significantly improves the precision compared to the basic SARIMA approach, even doubling it for some detection threshold values. The SARIMA-pruned approach outperforms the baseline approach in terms of precision, except for detection threshold values below 4 in decrease anomalies, for which the baseline attains slightly higher values.

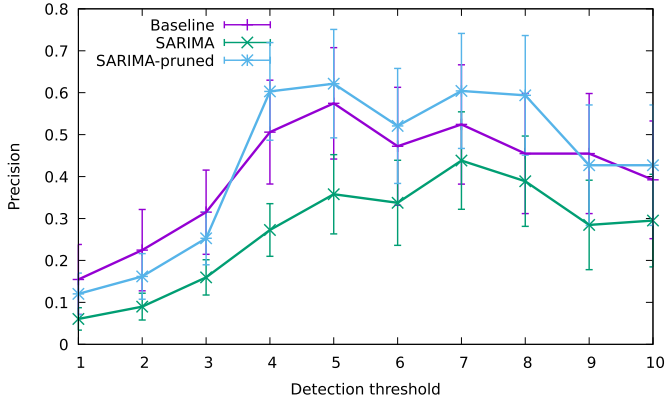
Fig. 8 shows the precision of the decrease and increase anomaly detection approaches, for a synthetic anomaly of a 40% reduction using different detection threshold values. As we can observe, the behavior is analogous to the 20% scenario, the SARIMA-pruned approach obtaining the highest precision, followed by the baseline and the basic SARIMA. In this case, the precision is slightly higher, since the anomaly is more severe and thus the number of TPs increases. In this case, the baseline only outperforms the SARIMA-pruned approach for detection thresholds lower than 2, further showing the robustness of our proposed approach.

Fig. 9 shows the recall of the decrease and increase anomaly detection approaches, for a synthetic anomaly of a 20% reduction using different detection threshold values. As we could expect, the recall of the approaches decreases when the detection threshold grows, since the number of FPs decreases faster than the number of TPs. However, the recall obtained by the SARIMA-pruned approach with a detection threshold of 1 is slightly lower than those obtained with higher thresholds. This is related to the huge number of FPs that occur for very low detection thresholds (e.g., 1), which result in the masking of some real anomalies due to the pruning process. The basic SARIMA approach always obtains the best recall result regardless of the detection threshold. On the other hand, for low detection thresholds (i.e., lower than 4) the baseline behaves slightly better than SARIMA-pruned, while detection thresholds higher than 4 make the SARIMA-pruned approach converge to the basic SARIMA outcome, thus also outperforming the baseline.

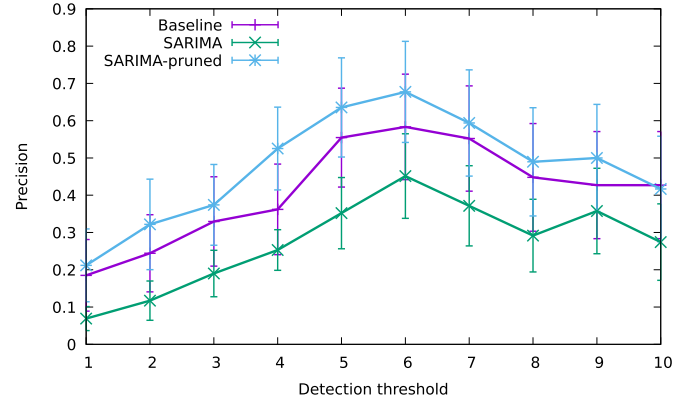
Fig. 10 shows the recall of the decrease and increase anomaly detection approaches, for a synthetic anomaly of a 40% reduction, using different detection threshold values. As we can observe, the behavior is analogous to the 20% scenario, the basic SARIMA obtaining the highest recall. In this case, we better notice the low recall for a detection threshold at 1, peaking at 2 and 3 for the decrease and increase anomalies, respectively. For detection thresholds higher than 3, SARIMA-pruned already outperforms the baseline attaining performance very close to the basic SARIMA.

Fig. 11 shows the F1-score of the decrease and increase anomaly detection approaches, for a synthetic anomaly of a 20% reduction, using different detection threshold values. As we could expect, the F1-score increases with the detection threshold up to a certain point. In the decrease anomalies case, it peaks at a detection threshold around 4 or 5. Further increasing the detection threshold reduces the F1-score. The maximum F1-score is achieved for a value of 6 in the increase anomalies case. The basic SARIMA attains the lowest F1-score, while the SARIMA-pruned attains the best, except for low detection thresholds below 4 in the decrease anomalies scenario, in which SARIMA-pruned is slightly outperformed by the baseline.

Finally, Fig. 12 shows the F1-score of the decrease and increase anomaly detection approaches, for a synthetic anomaly of a 40% reduc-

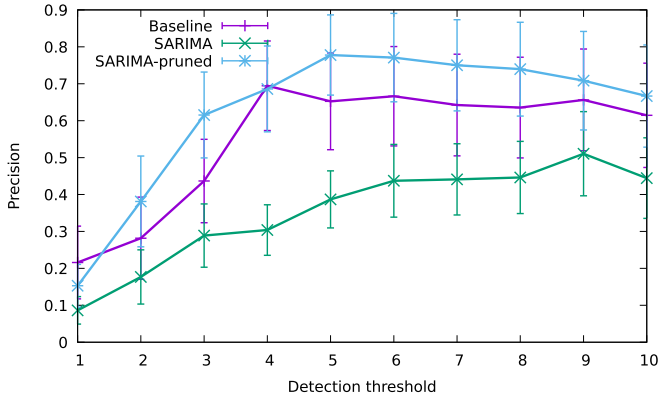


(a) Decrease anomalies

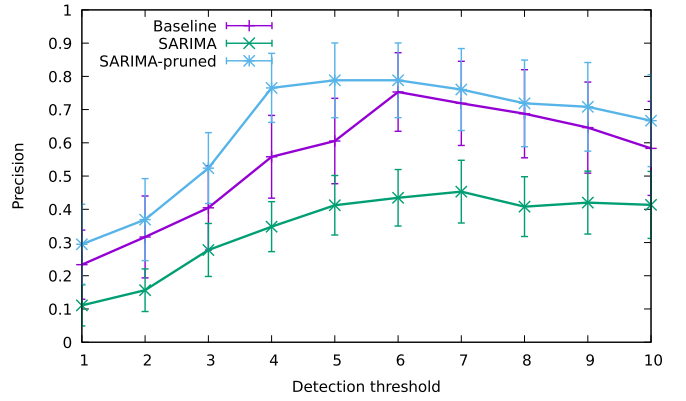


(b) Increase anomalies

Fig. 7. Precision of the anomaly detection approaches for a synthetic anomaly of a 20% reduction.

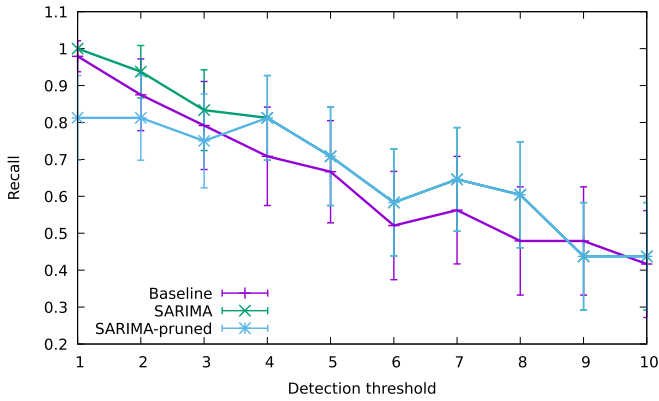


(a) Decrease anomalies

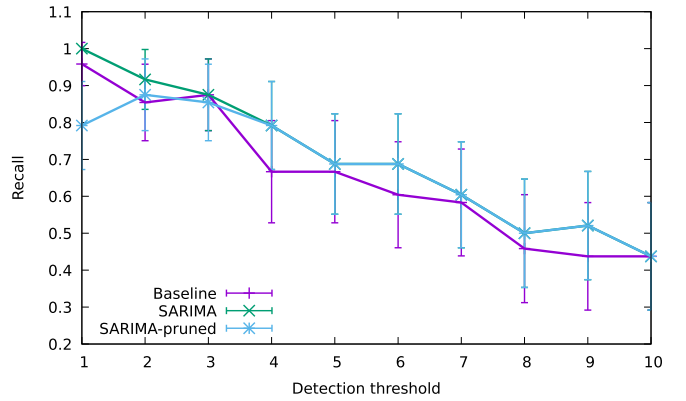


(b) Increase anomalies

Fig. 8. Precision of the anomaly detection approaches for a synthetic anomaly of a 40% reduction.



(a) Decrease anomalies



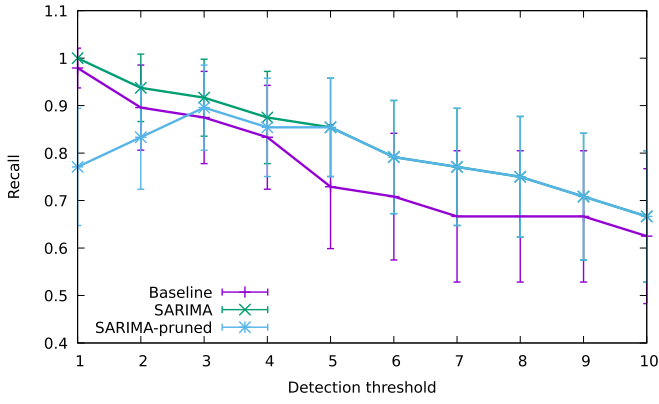
(b) Increase anomalies

Fig. 9. Recall of the anomaly detection approaches for a synthetic anomaly of a 20% reduction.

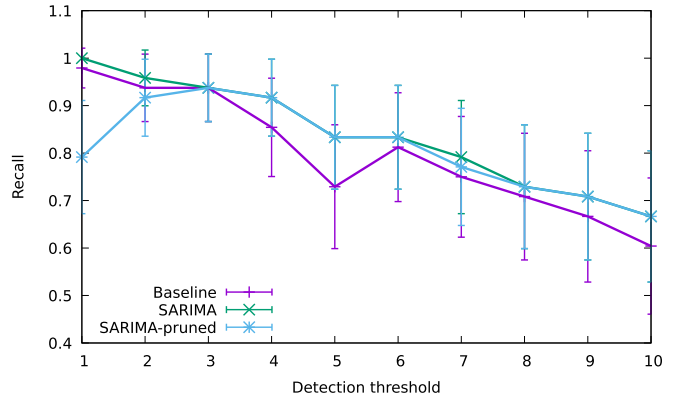
tion, using different detection threshold values. As we can observe, the behavior is again analogous to the 20% scenario, the basic SARIMA obtaining the best results, followed by the baseline and the basic SARIMA in the last place. We can also notice that the drops in the F1-score values attained is not so sharp as in the 20% anomaly, hence showing the robustness of the proposed detection method.

5.3. Anomaly detection capabilities: Real anomalies

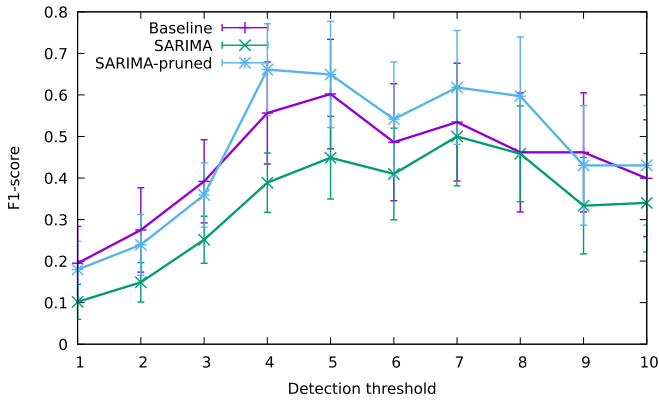
The total number of samples evaluated for each time series in this scenario is 8722. The dataset contains a set of real anomalies including the decrease anomaly and the corresponding increase anomaly, when available.



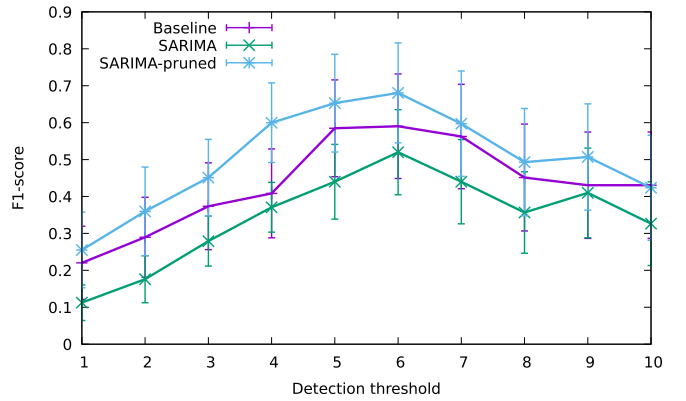
(a) Decrease anomalies



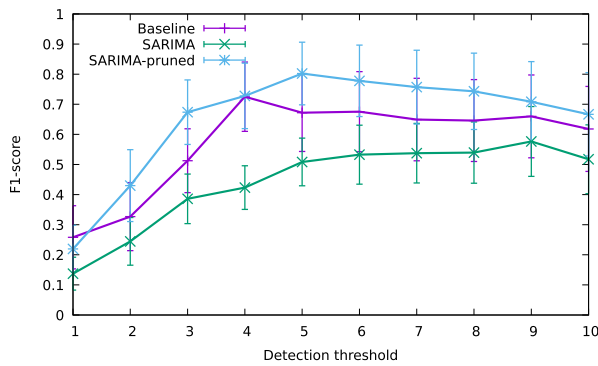
(b) Increase anomalies

Fig. 10. Recall of the anomaly detection approaches for a synthetic anomaly of a 40% reduction.

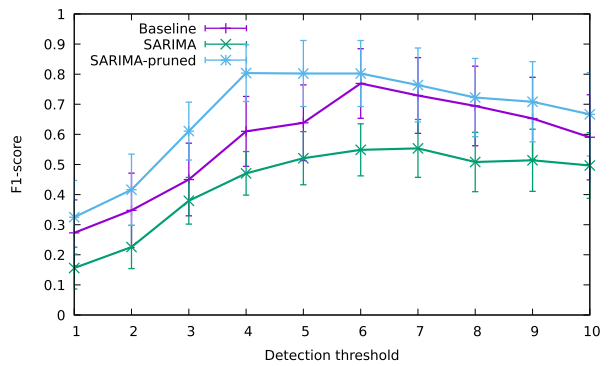
(a) Decrease anomalies



(b) Increase anomalies

Fig. 11. F1-score of the anomaly detection approaches for a synthetic anomaly of a 20% reduction.

(a) Decrease anomalies

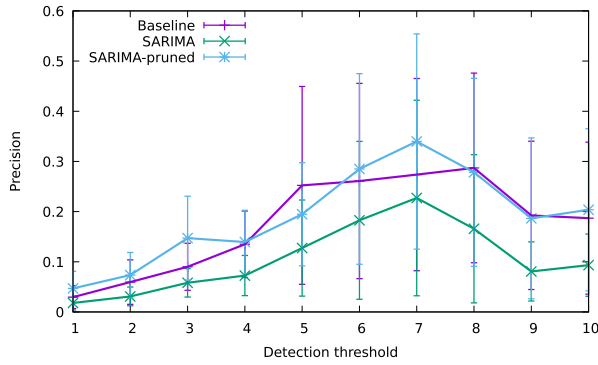


(b) Increase anomalies

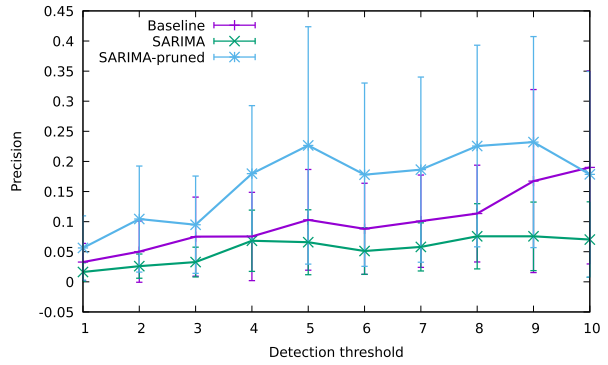
Fig. 12. F1-score of the anomaly detection approaches for a synthetic anomaly of a 40% reduction.

Fig. 13 shows the precision of the detection approaches for real anomalies, using different detection threshold values. As we can observe in the decrease anomalies, precision values grow until detection thresholds of 7 and 8 are reached, with SARIMA-based approaches and the baseline, respectively, and then they start to decrease. The basic SARIMA obtains the lowest precision, while SARIMA-pruned outperforms the baseline for every detection threshold except for a value of 5 in which the baseline slightly outperforms the SARIMA-pruned and 4, 8 and 9 in which both algorithms perform very similarly. The overall

best precision result was obtained with SARIMA-pruned for a detection threshold of 7. We can notice that the precision values are quite lower than in the previous experiments with synthetic anomalies. This is due to the overall number of samples analyzed, which in this case is 8722, while in the previous experiments it was 2016. This conveys an increase in the number of FPs, for a similar number of TPs, which, overall, results in a substantial decrease of the measured precision. The SARIMA-pruned is also the best approach for the increase anomalies, followed by the baseline and the basic SARIMA. The precision value is slightly

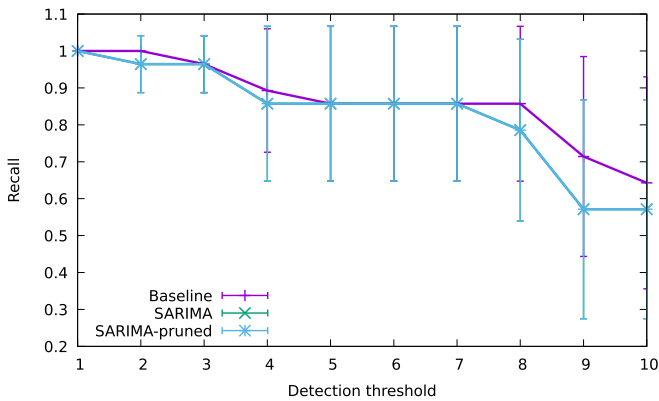


(a) Decrease anomalies

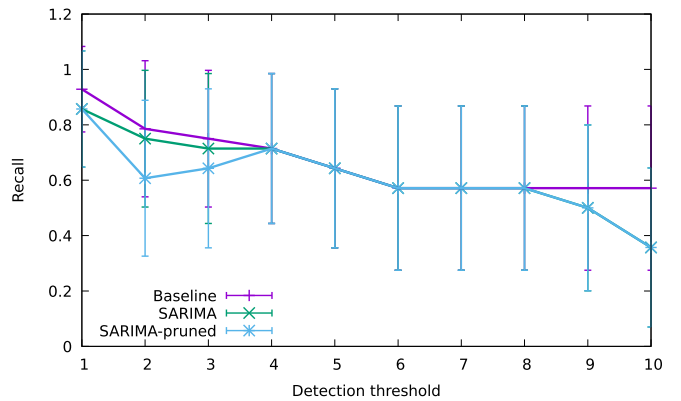


(b) Increase anomalies

Fig. 13. Precision of the detection approaches for real anomalies.



(a) Decrease anomalies



(b) Increase anomalies

Fig. 14. Recall of the detection approaches for real anomalies.

increasing with the detection threshold, with some fluctuations in the SARIMA-pruned approach.

Fig. 14 shows the recall of the detection approaches for real anomalies, using different detection threshold values. The recall values decrease when increasing the detection threshold, as expected. In this case, the recall obtained by both SARIMA-based approaches is nearly identical, with some differences in the increase anomalies for detection thresholds 2 and 3, where the recall of the SARIMA-pruned approach is slightly lower. For half of the detection thresholds the three approaches obtain the same recall, while for the rest the SARIMA-based approaches obtain a slightly lower recall.

Fig. 15 shows the F1-score of the detection approaches for real anomalies, using different detection threshold values. The F1-score behavior parallels the precision's behavior. For decrease anomalies, the SARIMA-pruned approach obtained the overall best result for a detection threshold of 7, outperforming the baseline for every threshold value except for 5, 9 and 10. For increase anomalies, the SARIMA-pruned obtained the overall best approach for a detection threshold of 9, outperforming the baseline for every detection threshold except 10.

5.4. Time complexity

As previously explained, we apply Kalman filters to the resulting SARIMA models to reduce the execution time of the prediction algorithm and enable real-time operation. Subsequently, we keep the Kalman filter updated with new data values instead of the SARIMA model. Although the results are not directly comparable, since the alternative using the

Table 3

Execution time (in milliseconds) for one-step prediction in R.

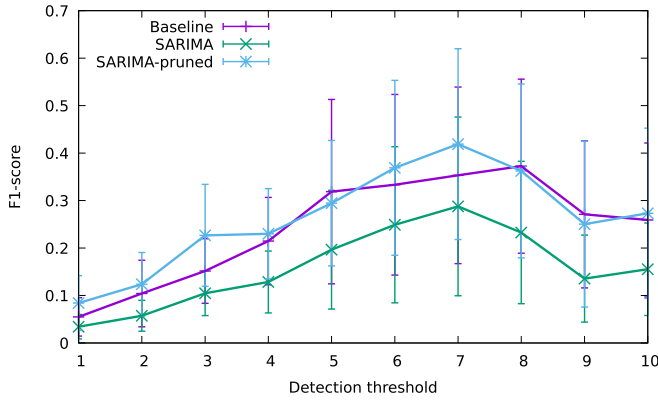
Strategy	Min	1st Qu	Median	Mean	3rd Qu	Max
ARIMA	643.3	646.7	649.8	651.8	653.6	777.4
Kalman	0.51	0.58	0.60	1.49	0.62	137.52

ARIMA model requires a loop that readjusts and updates the model with a historical record for each new sample, which is dependent on the R implementation, our experimental results in Table 3 indicate that updating the Kalman model is three orders of magnitude faster than updating the SARIMA model for one-step forecasts, producing exactly the same predicted values.¹ In detail, the mean execution time is more than 400 times faster when using Kalman, while median, first and third quartile values are more than 1000 times faster.

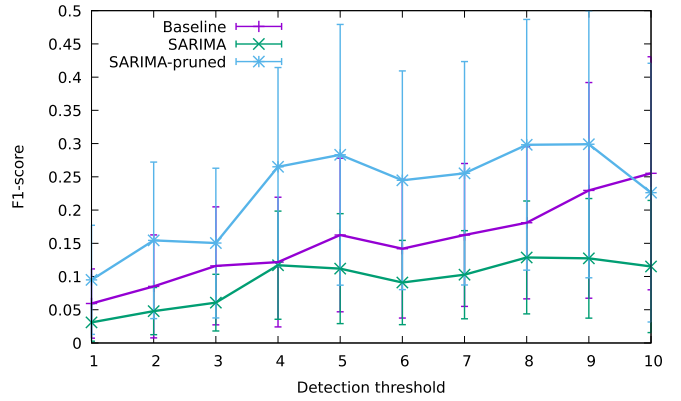
6. Discussion

In view of the results, the SARIMA-based approaches outperform the naïve baseline in terms of recall for the synthetic anomalies, while performing very similarly for the real anomalies. Although the basic SARIMA introduces more false positives than the baseline, the SARIMA-pruned approach significantly reduces them, thus increasing the precision. Note that the false positives introduced by the basic SARIMA

¹ Each call to the `Arima()` function receives the samples corresponding to the two previous days to provide adequate results.



(a) Decrease anomalies



(b) Increase anomalies

Fig. 15. F1-score of the detection approaches for real anomalies.

approach are directly related to the internal behavior of the algorithm: the prediction of the current sample is based on several previous values (depending on the different parameters of the algorithm), while in the baseline only the last sample has influence on the prediction. For this reason, when an anomaly takes place, the abnormal value is considered for calculating the predicted values for multiple samples, while in the baseline it is only used for one. The SARIMA-pruned approach ignores the residuals of the samples in which an abnormal value has been used for the prediction, ensuring that replicas and mirror anomalies are not identified as anomalies.

Furthermore, the SARIMA algorithm shows lower forecasting error compared to the baseline. Notably, the residual values approach white noise in absence of anomalies. However, the residuals of the baseline are strongly correlated with the time series values. For this reason, after filtering the replicas and mirror values, the SARIMA-based approaches outperform the baseline approach in anomaly detection capabilities.

The threshold for anomaly detection implies a trade-off between detecting many false-positive anomalies and identifying more real anomalies. According to our results, the most adequate value for the detection threshold, establishing a trade-off between precision and recall, lies between 4 and 7. A threshold value lower than 4 for the normalized residual values is expected to provide too many false positives (low precision), while values higher than 7 may only be able to detect very sharp anomalies (low recall). For example, the SARIMA-pruned approach is able to detect more than 85.4% synthetic anomalies of 40% in magnitude using a threshold of 4 and 77% when using a threshold of 7.

Overall, in view of our experimental results, the SARIMA-pruned approach is the best alternative, in terms of F1-score, recall and precision, for most detection threshold values.

The system has been successfully deployed in a nationwide network operator's infrastructure, demonstrating its capacity to concurrently monitor hundreds of network metrics in real-time. Subjective results from control dashboards have also been highly satisfactory.

7. Conclusions

Anomaly detection is crucial for the operation and maintenance of data communications infrastructures. In telecommunication networks, it is essential for reducing the operating expenses and address underlying problems in minimum time.

In this paper, we have proposed a SARIMA-based approach for real-time anomaly detection in operator networks. The proposed solution analyzes the time-series data generated by networking devices. Our proposal is composed of data normalization, forecasting and anomaly detection stages. Our experimental results, using real traffic traces provided

by a network operator and made publicly available, reveal that the proposed algorithm is able to reduce the number of false positives when compared to a baseline approach, while detecting a similar amount of actual anomalies. The trade-off between precision and recall can be fine-tuned in the proposed solution by adjusting the value of the anomaly detection threshold.

As future work we plan to extend the proposed solution to different network metrics of telecommunication operator networks, to further generalize the proposed approach.

CRedit authorship contribution statement

Pablo Fondo-Ferreiro: Writing - original draft, Visualization, Validation, Software, Investigation; **Miguel Rodelgo-Lacruz:** Writing - original draft, Visualization, Validation, Methodology, Formal analysis, Data curation; **Francisco Javier González-Castaño:** Writing - review & editing, Supervision, Project administration, Funding acquisition, Conceptualization; **Felipe Gil-Castiñeira:** Writing - review & editing, Supervision, Resources, Project administration, Methodology; **Carlos Zamorano-Pinal:** Writing - review & editing, Supervision, Resources, Project administration, Conceptualization; **David Candal-Ventureira:** Writing - review & editing, Validation, Investigation, Data curation.

Data availability

We have shared the data in a Zenodo repository and the link is referenced in the manuscript.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgement

This work was supported in part by Xunta de Galicia, Spain, under Grant ED481B-2022-019 and Grant ED431C 2022/04; in part by the Ministerio de Ciencia, Innovación y Universidades, Spain, under Grant PID2023-148214OB-C22; in part by the R&D&I project "Creation of a network of laboratories and demonstration centers for research and innovation in cybersecurity solutions," funded by the European Union - NextGenerationEU, the Recovery, Transformation and Resilience Plan, and INCIBE; and in part by the Universidade de Vigo/CISUG for Open Access Charge.

References

- [1] A. Blázquez-García, A. Conde, U. Mori, J.A. Lozano, A review on outlier/anomaly detection in time series data, *ACM Comput. Surv. (CSUR)* 54 (3) (2021) 1–33.
- [2] A.A. Cook, G. Misrlı, Z. Fan, Anomaly detection for IoT time-series data: a survey, *IEEE Internet Things J.* 7 (7) (2019) 6481–6494.
- [3] M. Ahmed, A.N. Mahmood, J. Hu, A survey of network anomaly detection techniques, *J. Netw. Comput. Appl.* 60 (2016) 19–31.
- [4] Y. Zhang, Z. Ge, A. Greenberg, M. Roughan, Network anomography, in: *Proceedings of the 5th ACM SIGCOMM Conference on Internet Measurement*, 2005, pp. 30.
- [5] A.H. Yaacob, I.K.T. Tan, S.F. Chien, H.K. Tan, Arima-based network anomaly detection, in: *2010 Second International Conference on Communication Software and Networks*, IEEE, 2010, pp. 205–209.
- [6] P. Cortez, M. Rio, M. Rocha, P. Sousa, Multi-scale internet traffic forecasting using neural networks and time series methods, *Expert Syst.* 29 (2) (2012) 143–155.
- [7] M.F. Zhani, H. Elbiaze, F. Kamoun, Analysis and prediction of real network traffic, *J. Netw.* 4 (9) (2009) 855–865.
- [8] R. Madan, P.S. Mangipudi, Predicting computer network traffic: a time series forecasting approach using DWT, ARIMA and RNN, in: *2018 Eleventh International Conference on Contemporary Computing (IC3)*, IEEE, 2018, pp. 1–5.
- [9] Y. Yu, J. Wang, M. Song, J. Song, Network traffic prediction and result analysis based on seasonal ARIMA and correlation coefficient, in: *2010 International Conference on Intelligent System Design and Engineering Application*, 1, IEEE, 2010, pp. 980–983.
- [10] A. Teker, A.H. Örneş, B. Canberk, Network bandwidth usage forecast in content delivery networks, in: *2020 International Conference on Broadband Communications for Next Generation Networks and Multimedia Applications (CoBCom)*, IEEE, 2020, pp. 1–6.
- [11] T.P. Oliveira, J.S. Barbar, A.S. Soares, Computer network traffic prediction: a comparison between traditional and deep learning neural networks, *Int. J. Big Data Intell.* 3 (1) (2016) 28–37.
- [12] S. Nihale, S. Sharma, L. Parashar, U. Singh, Network traffic prediction using long short-term memory, in: *2020 International Conference on Electronics and Sustainable Communication Systems (ICESC)*, IEEE, 2020, pp. 338–343.
- [13] R. Vinayakumar, K.P. Soman, P. Poornachandran, Applying deep learning approaches for network traffic prediction, in: *2017 International Conference on Advances in Computing, Communications and Informatics (ICACCI)*, IEEE, 2017, pp. 2353–2358.
- [14] H. Liu, H.-q. Tian, Y.-f. Li, Comparison of two new ARIMA-ANN and ARIMA-Kalman hybrid methods for wind speed prediction, *Appl. Energy* 98 (2012) 415–424.
- [15] D.-w. Xu, Y.-d. Wang, L.-m. Jia, Y. Qin, H.-h. Dong, Real-time road traffic state prediction based on ARIMA and Kalman filter, *Front. Inf. Technol. Electron. Eng.* 18 (2) (2017) 287–302.
- [16] S. Kang, S. Lee, Y. Won, B. Seong, On-line prediction of non-stationary variable-bit-rate video traffic, *IEEE Trans. Signal Process.* 58 (3) (2009) 1219–1237.
- [17] P.J. Brockwell, R.A. Davis, *Time Series: Theory and Methods*, Springer Science & Business Media, 2009.
- [18] R.J. Hyndman, G. Athanasopoulos, *Forecasting: Principles and Practice*, OTexts, 2018.
- [19] H. Wold, A study in the analysis of stationary time series, Ph.D. thesis, Almqvist & Wiksell, 1938.
- [20] T. Bollerslev, Generalized autoregressive conditional heteroskedasticity, *J. Econom.* 31 (3) (1986) 307–327.
- [21] G.E.P. Box, D.R. Cox, An analysis of transformations, *J. R. Stat. Soc. Ser. B (Methodol.)* 26 (2) (1964) 211–243.
- [22] V.M. Guerrero, Time-series analysis supported by power transformations, *J. Forecast.* 12 (1) (1993) 37–48.
- [23] S.S. Shapiro, M.B. Wilk, An analysis of variance test for normality (complete samples), *Biometrika* 52 (3/4) (1965) 591–611.
- [24] J.J. Filliben, The probability plot correlation coefficient test for normality, *Technometrics* 17 (1) (1975) 111–117.
- [25] R. Morris, D. Lin, Variance of aggregated web traffic, in: *Proceedings IEEE INFOCOM 2000. Conference on Computer Communications. Nineteenth Annual Joint Conference of the IEEE Computer and Communications Societies (Cat. No. 00CH37064)*, 1, IEEE, 2000, pp. 360–366.
- [26] R. Kawahara, K. Ishibashi, T. Asaka, K. Ori, A method of IP traffic management using TCP flow statistics, in: *IEEE Global Telecommunications Conference, 2004. GLOBECOM'03*, 7, IEEE, 2003, pp. 4059–4063.
- [27] R. Kawahara, K. Ishibashi, A. Takuya, S. Sumita, T. Abe, A method of bandwidth dimensioning and management using flow statistics [IP networks], in: *IEEE Global Telecommunications Conference, 2004. GLOBECOM'04*, 2, IEEE, 2004, pp. 670–674.
- [28] K. Ishibashi, R. Kawahara, T. Asaka, M. Aida, S. Ono, S. Asano, Detection of TCP performance degradation using link utilization statistics, *IEICE Trans. Commun.* 89 (1) (2006) 47–56.
- [29] T. Karagiannis, M. Molle, M. Faloutsos, A. Broido, A nonstationary poisson view of internet traffic, in: *IEEE INFOCOM 2004*, 3, IEEE, 2004, pp. 1558–1569.
- [30] D. Kwiatkowski, P.C.B. Phillips, P. Schmidt, Y. Shin, Testing the null hypothesis of stationarity against the alternative of a unit root: how sure are we that economic time series have a unit root?, *J. Econom.* 54 (1-3) (1992) 159–178.
- [31] R.J. Hyndman, G. Athanasopoulos, C. Bergmeir, G. Caceres, L. Chhay, M. O'Hara-Wild, F. Petropoulos, S. Razbash, E. Wang, Package 'forecast', [Online] <https://cran.r-project.org/web/packages/forecast/forecast.pdf> (2020).
- [32] V. Paxson, S. Floyd, Wide-area traffic: the failure of poisson modeling, *IEEE/ACM Trans. Netw.* 3 (3) (1995) 226–244.
- [33] P. Fondo-Ferreiro, M. Rodelgo-Lacruz, F.J. González-Castaño, F. Gil-Castiñeira, D. Candal-Ventureira, Network operator kpis time series dataset, 2023, <https://doi.org/10.5281/zenodo.8147768>
- [34] A. Lavin, S. Ahmad, Evaluating real-time anomaly detection algorithms—the numenta anomaly benchmark, in: *2015 IEEE 14th International Conference on Machine Learning and Applications (ICMLA)*, IEEE, 2015, pp. 38–44.
- [35] A. De Myttenaere, B. Golden, B. Le Grand, F. Rossi, Mean absolute percentage error for regression models, *Neurocomputing* 192 (2016) 38–48.